



Ellen

Andrew

Jayce

Shwei

Disha

quentin

Cody

Jonh

Zoe

Trisha

Gaya

Melw

Tolu

Daniel

21-671: Computational Linear Algebra

Lecture #3

Updated August 26, 2025

Sources of Error in Computations

In general, we compute solutions to mathematical problems on computers by replacing infinite processes with finite processes. This means we are almost always **approximating** the true mathematics, and often we must also approximate the data used in the computation.

Approximations made **before** computation:

- Simplifying models of physical phenomena
- Precision of empirical measurements
- Previous computations

Approximations made **during** computation:

- **Truncation** or **discretization**: i.e. replace an infinite series with a finite sum, e.g. the Taylor series
↳ e.g. example
- **Rounding**: replacing a real number with its floating-point representation
↳ for example

Error definitions: Here we denote x as the true or exact value, $\hat{x}$ as the approximate value.

True value minus approximate value gives you error $\rightarrow$ absolute error: $|x - \hat{x}|$
 relative error: $\frac{|x - \hat{x}|}{|x|}$ ($x \neq 0$) $\rightarrow$ percentage error

We can use the following formula to get the gist of how to calculate the approximate value:

$$\text{approximate value} = (\text{true value})(1 \pm \text{relative error})$$

Note that the plus/minus will depend on whether the true value is positive/negative and if the approximate value is greater than/less than the true value.

Computational Error

Let f be a mathematical function, x be the true input, and $\hat{x}$ be the approximate input. If we can compute f exactly, the error in the computation is equal to what is called the:

$$\text{propagated data error: } |f(x) - f(\hat{x})|$$

If, on the other hand, we must approximate f with $\hat{f}$, then our output is $\hat{f}(\hat{x})$.

$$|f(x) - \hat{f}(\hat{x})| \leq \underbrace{|f(x) - f(\hat{x})|}_{\text{prop data error}} + \underbrace{|f(\hat{x}) - \hat{f}(\hat{x})|}_{\text{comp error}}$$

Computational error may be due to:

$$|f(x) - \hat{f}(\hat{x})| = |f(x) - f(\hat{x}) + f(\hat{x}) - \hat{f}(\hat{x})| \leq |f(x) - f(\hat{x})| + |f(\hat{x}) - \hat{f}(\hat{x})|$$

this is zero

- **Truncation error:** difference between true result (for the true input) and the result produced by a given algorithm using exact arithmetic
 - E.g., the difference between the exact value of e vs. the fifth-order Taylor series approximation $\sum_{n=0}^5 \frac{1}{n!}$ of e
- **Rounding error:** difference between the result produced by a given algorithm using **exact arithmetic** and the result produced by the same algorithm using **finite-precision, rounded arithmetic**
 - E.g., the difference between the value of $\sum_{n=0}^5 \frac{1}{n!}$ and the value of same series rounded to three decimals (i.e. 2.708)

These both play a role, but usually one is dominated by another in a particular computation.

- Rounding error tends to dominate in purely algebraic problems with finite (exact) solution algorithms
- Truncation error tends to dominate in problems that require infinite solution processes

Note, however, that this isn't always the case. For instance, a truncated Taylor series may have relatively small truncation error if enough terms are used (e.g. 20), but if the computer rounds the result to only a few decimal places (e.g. 4), the rounding error will be much larger.

→ true value of e - value of e given by Taylor series
Example 1. Estimate the truncation error in approximating e by the fifth-order Taylor series $\sum_{n=0}^5 \frac{1}{n!}$, to 6 decimal places.

$$\sum_{n=0}^5 \frac{1}{n!} = \frac{1}{1} + \frac{1}{1} + \frac{1}{2} + \frac{1}{6} + \frac{1}{24} + \frac{1}{120} = \frac{326}{120} = 2.716$$

Now, the value of $e \approx 2.718281828459415...$
 Thus, the truncation error is approx:

Truncation error: $\left| e - \sum_{n=0}^5 \frac{1}{n!} \right| \approx |2.718281828 - 2.716| \approx 0.0016$

Example 2. Find the rounding error in approximating e by the fifth-order Taylor series $\sum_{n=0}^5 \frac{1}{n!}$, rounded down to 4 decimal places.

$$\sum_{n=0}^5 \frac{1}{n!} - \text{round}\left(\sum_{n=0}^5 \frac{1}{n!}, 4\right)$$

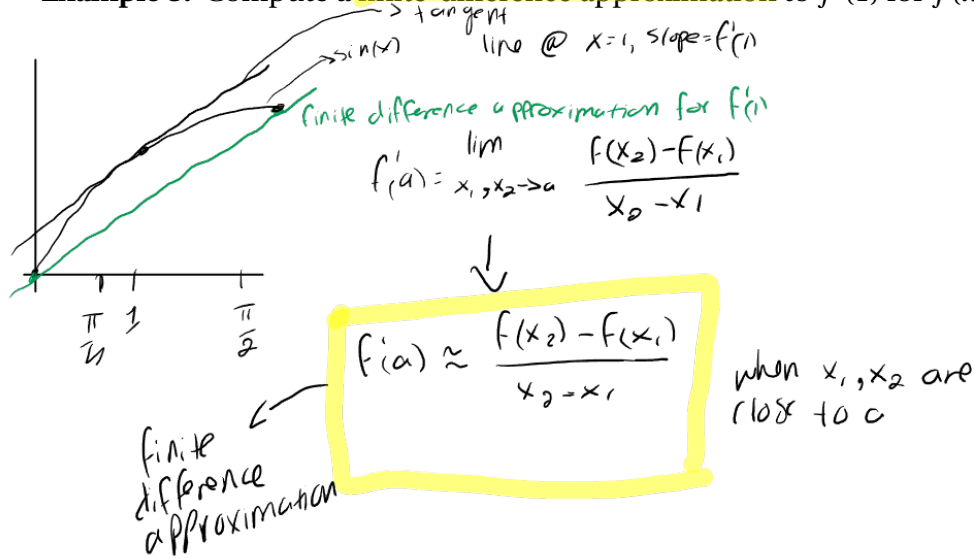
$$= |2.716 - 2.7167|$$

$$= 2.7167 - 2.716 = 0.0007$$

→ difference between the series and rounding that series up to 4 decimal places

★ .70 - .66 (6 repeating) is equal to 3 because .66... = 0.6666... ≈ .67

Example 3. Compute a **finite-difference approximation** to $f'(1)$ for $f(x) = \sin x$ with 4 decimal places.



* Notice that 1 is in between 0 and $\frac{\pi}{2} \approx 1.57$

* So, choose points $x_1 = 0$ & $x_2 = \frac{\pi}{2}$ as the two x -values for our secant line

$$\sin'(1) = \frac{\sin(\frac{\pi}{2}) - \sin(0)}{\frac{\pi}{2} - 0} = \frac{1}{\frac{\pi}{2}} = \frac{2}{\pi} \approx 0.6366$$

Another one: $x_0 = \frac{\pi}{3} \approx 1.05$, $x_1 = \frac{\pi}{6} \approx 0.52$

$$\sin'(1) = \frac{\sin(\frac{\pi}{3}) - \sin(\frac{\pi}{6})}{\frac{\pi}{3} - \frac{\pi}{6}} = \frac{\frac{\sqrt{3}}{2} - \frac{1}{2}}{\frac{\pi}{3} - \frac{\pi}{6}} = \frac{3\sqrt{3} - 3}{\pi} \approx 0.6991$$

+ real value: $\sin'(1) \approx 0.5403$

Forward and Backward Error

Often we will lump all of the computational errors together and treat them as error in the input data. One analogy that may help to explain this, is that we are “placing all of the blame” for any error solely on the input, despite much of that error coming from the computation itself! This means we can write $f(\hat{x})$ by choosing $\hat{x}$ appropriately.

Task: Compute $y = f(x)$. **Result:** $\hat{y} = \hat{f}(x) = f(\hat{x})$

pretending that the error in the computation is the same as the error coming from the input ($\hat{x}$)

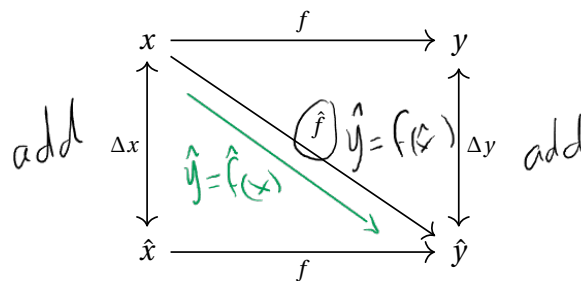
- Backward error:** error in initial data required to explain all of the error in the computed result:

$$|\Delta x| = |\hat{x} - x|$$

- Forward error:** discrepancy between true and computed value:

$$|\Delta y| = |\hat{y} - y|$$

We also have the **signed forward error** $\Delta y = \hat{y} - y$ and the **signed backward error** $\Delta x = \hat{x} - x$, which can be useful to find in some cases where we need to know whether the approximate value is above or below the true value. However, unless otherwise specified, forward and backward error will refer to the **absolute forward error** $|\Delta y|$ and **absolute backward error** $|\Delta x|$.



Example 4. Suppose you wanted to approximate $y = f(x) = \cos x$ by the truncated series $\hat{y} = \hat{f}(x) = 1 - \frac{x^2}{2}$. Find the formulas for the signed forward and backward error in terms of x . Then estimate the signed forward and backward error for $x = 1$ to four decimal places.

$$\begin{aligned} \Delta y &= \hat{y} - y \quad (\text{signed forward error}) \\ &= 1 - \frac{x^2}{2} - \cos x \end{aligned}$$

$$\begin{aligned} x &= 1 \\ \Delta y &= 1 - \frac{1}{2} - \cos 1 \approx 0.5 - 0.5403 \\ &\approx -0.0403 \approx -7.5\% \end{aligned}$$

$$\begin{aligned} \Delta x &= \hat{x} - x \quad (\text{signed backward error}) \\ \text{How do we find } \hat{x}? \\ \text{use formula } \hat{f}(x) &= f(\hat{x}) \end{aligned}$$

$$\frac{-0.0403}{0.5403} = \text{relative error}$$

$$1 - \frac{x^2}{2} = \cos(\hat{x})$$

$$\hat{x} = \cos^{-1}\left(1 - \frac{x^2}{2}\right)$$

$$\begin{aligned} \Delta x &= 1.0472 - 1 \\ &\approx 0.0472 \\ &\approx 4.7\% \end{aligned}$$

relative error
error · original
thing w/o
error

$$\Delta x = \hat{x} - x = \cos^{-1}\left(1 - \frac{x^2}{2}\right) - x$$

21-671: Computational Linear Algebra

Lecture #4

Updated September 2, 2025

Sensitivity and Conditioning

When an algorithm produces an inaccurate solution to a mathematical problem, it may not necessarily be due to an ill-conceived algorithm. Instead, it may be an inherent property of the mathematical problem itself.

Definition: A mathematical problem is **sensitive or ill-conditioned** if small changes in input data can produce large changes in output; it is **insensitive or well-conditioned** if relative changes in input produce similar relative changes in output. (Sensitivity is usually a **qualitative** measure, while conditioning is a **quantitative** measure.)

Definition: The **condition number** of a mathematical problem is the ratio of the relative change in output to the relative change in input.

ratio of change in output to change in input

Using the above definition, we can come up with the following formula:

$$\text{condition number} = \frac{|\text{relative forward error}|}{|\text{relative backward error}|} = \frac{\frac{|f(\hat{x}) - f(x)|}{|f(x)|}}{\frac{|\hat{x} - x|}{|x|}} = \frac{\frac{|\hat{y} - y|}{|y|}}{\frac{|\hat{x} - x|}{|x|}} = \left| \frac{\frac{\Delta y}{y}}{\frac{\Delta x}{x}} \right| = \left| \frac{\Delta y}{\Delta x} \cdot \frac{x}{y} \right|$$

Similarly, we also have the formula:

$$\text{condition number} \cdot |\text{relative backward error}| = |\text{relative forward error}|$$

Often, rather than knowing what the condition number is explicitly equal to, we can only bound the condition number from above. For instance:

Example 1. Consider the problem of evaluating a differentiable function $f: \mathbb{R} \rightarrow \mathbb{R}$. Give an approximation of the condition number of f using the derivative $f'(x)$.

what does a small change in input do to the output?

Recall from Lecture 3:

relative forward error: $\left| \frac{f(x+\Delta x) - f(x)}{f(x)} \right|$ when Δx is small

not the same

interchange

$$= \left| \frac{f(x+\Delta x) - f(x)}{\Delta x} \cdot \frac{\Delta x}{f(x)} \right| \approx \left| \frac{f'(x) \cdot \Delta x}{f(x)} \right|$$

$\approx f'(x)$

condition number:

$$\frac{\left| \frac{f'(x) \Delta x}{f(x)} \right|}{\left| \frac{\Delta x}{x} \right|} = \frac{x f'(x)}{f(x)}$$

Example 2. For the following, use the derivative estimate from the solution of Example 1 to find the condition number of f .

(a) $f(x) = \sqrt{x}$

(b) $f(x) = x^5$

(c) $f(x) = e^x$

a) condition number $\approx \left| \frac{x f'(x)}{f(x)} \right| = \left(\frac{x (\frac{1}{2\sqrt{x}})}{\sqrt{x}} \right) = \left| \frac{x}{2x} \right| = \frac{1}{2}$

$\rightarrow$ a 1% change in x leads to 0.5% change in y

$f'(x) = \frac{1}{2\sqrt{x}}$

b) $\frac{x 5x^4}{x^5} = 5 \rightarrow$ a 1% change in x leads to a 5% change in y

c) $\frac{x e^x}{e^x} = x \rightarrow$ a 1% change in x leads to an $x\%$ change in y

Stability and Accuracy

If we can speak of the sensitivity of mathematical problems, we may similarly speak of the **stability** of algorithms used to solve those problems.

Definition: An algorithm is **stable** if the result it produces is relatively insensitive to the perturbations due to approximations made during the computation.

Definition: Accuracy is the closeness of a computed solution to the true solution of the problem under consideration.

Note that stability does not necessarily imply accuracy! But stability and well-conditioning does imply accuracy.

Inaccuracy can result from:

- A stable algorithm for an ill-conditioned problem.
- An unstable algorithm for a well-conditioned problem.

$$\sqrt{2} = 1.414213562\dots$$

Example 3 (Babylonian algorithm). Consider the following algorithm for finding the value of $\sqrt{2}$ (which is a well-conditioned problem, as shown in Example 2, (a)). Starting with an initial guess x_0 , for each $n = 0, 1, 2, \dots$ we compute the following:

$$x_{n+1} = \frac{x_n + \frac{x_n}{2}}{2}$$

Is this algorithm stable?

let's start with an initial guess of 1.4

$$1.4^2 = 1.96 \approx 2$$

so it's a good guess

$$x_0 = 1.4$$

$$x_1 = \frac{x_0 + \frac{x_0}{2}}{2} = \frac{1.4 + \frac{1.4}{2}}{2} = 1.4142857\dots$$

(Another guess: 1.42
perturbs 1.4 by 0.02)

$$x_2 = 1.414213564\dots$$

accurate to 8 decimal places

$$x_0 = 1.42$$

$$x_1 = 1.41422535\dots$$

$$x_2 = 1.41421356242\dots$$

9 decimals

the algorithm is stable!

Example 4. Consider the following alternative algorithm for finding the value of $\sqrt{2}$. Starting with an initial guess x_0 , for each $n = 0, 1, 2, \dots$ we compute the following:

$$x_{n+1} = (x_n^2 - 2)^2 + x_n$$

Is this algorithm stable?

$$x_0 = 1.4$$

$$x_1 = 1.4016$$

$$x_2 = 1.4028614$$

$\vdots$

$$x_{1000000} = 1.41421\dots$$

5 decimals

$$x_0 = 1.42$$

$$x_1 = 1.42026896$$

$$x_2 = 1.42056\dots$$

$\vdots$

$$x_{17} = 7286.2284\dots$$

This algorithm is unstable! 

21-671: Computational Linear Algebra**Lecture #5**

Updated September 7, 2025

Binary numbers

Recall the meaning of decimal numbers. A decimal number is simply a sum of digits (~~with 10 to choose from~~) times powers of 10, with 10 digits to choose from. For example, we can expand the number 124.456 in the following way:

$$123.456 = 1 \times 10^2 + 2 \times 10^1 + 3 \times 10^0 + 4 \times 10^{-1} + 5 \times 10^{-2} + 6 \times 10^{-3}.$$

And so, a decimal number may be written as follows:

$$(\dots d_3 d_2 d_1 d_0 . d_{-1} d_{-2} d_{-3} \dots)_{10}, \quad d_i \in \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}.$$

$$\sum d_i 10^i$$

This number is equal to the following decimal expansion:

$$\dots + d_3 10^3 + d_2 10^2 + d_1 10^1 + d_0 10^0 + d_{-1} 10^{-1} + d_{-2} 10^{-2} + d_{-3} 10^{-3} + \dots$$

Binary numbers are similar, the difference being that a base of 2 is used rather than a base of 10: a binary number is a sum of digits (with 2 to choose from) times powers of 2. Thus, any binary number can be written as:

$$(\dots b_3 b_2 b_1 b_0 . b_{-1} b_{-2} b_{-3} \dots)_2, \quad b_i \in \{0, 1\}.$$

This number is equal to the following binary expansion:

$$\dots + b_3 2^3 + b_2 2^2 + b_1 2^1 + b_0 2^0 + b_{-1} 2^{-1} + b_{-2} 2^{-2} + b_{-3} 2^{-3} + \dots$$

Example 1. Convert the following to binary by writing an arithmetic expansion: 4, 10, 3/4.

$$4 = (1) \times 2^2 + (0) \times 2^1 + (0) \times 2^0 = (100)_2$$

$$10 = 1 \times 2^3 + 1 \times 2^1 = (1010)_2$$

$$\frac{3}{4} = \frac{1}{2} + \frac{1}{4} = 1 \times 2^{-1} + 1 \times 2^{-2} = (0.11)_2$$

integer → Trick: recursively divide by 2 and record remainders

Example 2. Convert 39.7 to binary.

Integer Part:

Fractional: multiply by 2
 $39 \div 2 = 19 \text{ remainder } 1$

$$19 \div 2 = 9 \text{ remainder } 1$$

$$9 \div 2 = 4 \text{ remainder } 1$$

$$4 \div 2 = 2 \text{ remainder } 0$$

$$2 \div 2 = 1 \text{ remainder } 0$$

$$1 \div 2 = 0 \text{ remainder } 1$$

$$(39)_{10} = (100111011)_2$$

Fractional part:

$$0.7 \times 2 = 1.4 \rightarrow 1$$

$$0.4 \times 2 = 0.8 \rightarrow 0$$

$$0.8 \times 2 = 1.6 \rightarrow 1$$

$$0.6 \times 2 = 1.2 \rightarrow 1$$

$$0.2 \times 2 = 0.4 \rightarrow 0$$

$$0.4 \times 2 = 0.8 \rightarrow 0$$

$$0.8 \times 2 = 1.6 \rightarrow 1$$

$$0.6 \times 2 = 1.2 \rightarrow 1$$

$$0.2 \times 2 = 0.4 \rightarrow 1$$

Example 3. Convert $(1011)_2$ from binary to decimal.

$$1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 8 + 0 + 2 + 1 = 11 \quad (0.7)_{10} = (0.10110)_2$$

Example 4. Convert $(0.1011)_2$ from binary to decimal.

Repeating fractional part = harder, vgs twice!

$$x = (0.1011)_2 = (0.1011 \ 1011 \ 1011 \dots)_2$$

$$(2^4)x = (1011.1011 \ 1011 \ 1011 \dots)_2$$

$$\hookrightarrow 2^4 x = (1011)_2$$

$$15x = (1011)_2 = 11$$

$$15x = 11$$

$$x = \frac{11}{15}$$

moving the decimal 4 to the right is the same thing as multiplying it by 2^4

$(2^4 - 1)x$
 example 3

if it's not repeating

rule for decimal to binary

- Divide whole numbers by 2 & record remainder
- multiply fractional parts (0.n; n being an number) by 2

for binary to decimal

$$\rightarrow (1011)_2 \rightarrow \begin{matrix} 1 & 1 & 1 & 1 \\ 3 & 2 & 1 & 0 \end{matrix}$$

$$1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 11$$

Example 5. Convert $(0.10\overline{101})_2$ from binary to decimal.

$$x = (0.10\overline{101})_2$$

$$2^2 x = (10.\overline{101})_2 = (10.)_2 + (.10\overline{1})_2 = 2 + y$$

$$\text{eqn 1.} \rightarrow 4x = 2 + y \rightarrow y = (.10\overline{1})_2 \rightarrow 2^3 y = (101.\overline{101})_2$$

$$2^3 y = (101.\overline{101})_2$$

$$2^3 y - y = (101)_2 = 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 5$$

$$(2^3 - 1)y = 5$$

$$7y = 5 \rightarrow y = \frac{5}{7}$$

Chopping and rounding

$$4x = 2 + y = 2 + \frac{5}{7} = \frac{19}{7}$$

$$x = \frac{19/7}{4} = \frac{19}{28}$$

Just as with decimal numbers, we may chop or round binary numbers as well. We will use the example number of $(0.10\overline{1})_2$ to illustrate this.

- $\rightarrow$ Chopping is the same as truncation
- Chopping: truncate the binary expression

– E.g. $(0.10\overline{1})_2 \approx (0.101)_2$ when truncated to 3 digits

- Rounding (to nearest): round up if the first bit after the mantissa (the significant digits we wish to keep) is 1.

– E.g. $(0.10\overline{1})_2 \approx (0.110)_2$ when rounded to 3 digits

3 sig. figs. = mantissa is 3 sig figs

Round up if the next digit is 1 and round down if the

$(0.101\overline{101} \dots)$ next digit is 0

21-671: Computational Linear Algebra

Lecture #6

Updated September 9, 2025

Floating Point Number Systems

Most scientific computations are performed using floating-point arithmetic. A t -digit floating point number in base β has the form $\pm m \times \beta^e$, where m is a t -digit fraction called the **mantissa**, β is the **base**, and e is the **exponent** (not to be confused with Euler's number $e \approx 2.71$). If we require the first digit of the mantissa to be nonzero, then we say the number is **normalized**. The number of digits in the mantissa is called the **precision**. Most of today's computers use the IEEE 754 Floating-Point Standard, in which $t = 52$ and the exponent has 11 digits for **double-precision** (i.e., 64 bit) numbers.

In binary ($\beta = 2$), all nonzero numbers are normalized and the first bit is stored **implicitly**:

Definition: A **machine number** is a real number that can be represented exactly in a computer.

Definition: For a floating-point system, the **machine epsilon**, ϵ_{mach} , is the distance between 1 and the smallest number greater than 1. For IEEE double, $\epsilon_{\text{mach}} = 2^{-52} \approx 2.2204 \times 10^{-16}$.

Definition: For any real number x , let $\text{fl}(x)$ represent the double-precision machine number obtained by the IEEE rounding rules. (Then a bound on the relative rounding error can be given by $\frac{|\text{fl}(x) - x|}{|x|} \leq \epsilon_{\text{mach}}$.) We often write $\text{fl}(x) = x(1 + \delta)$, where $|\delta| \leq \epsilon_{\text{mach}}$.

The result of floating-point computation must usually be **rounded** to a machine number for storage in the computer.

Example 1. Compute $\text{fl}(x)$ for $x = 1, 0.25, 0.2$.

1) $\text{fl}(1) = (1)_2$
 $(1)_{10} = (1)_2$
 $= 1.0 \dots 0 \times 2^0$
 52 digits

2) $\text{fl}(0.25) = (0.25)_{10} = (0.01)_2 = 1.0 \dots 0 \times 2^{-2}$
 52

3) $\text{fl}(0.2) = (0.2)_{10} = (0.0011)_2$

$2 \times 2 = 4 + 0$

$4 \times 2 = 8 + 0$

$8 \times 2 = 16 + 0$

$16 \times 2 = 32 + 0$

multiply by 2^{-3}

$1.1001100110011 \dots 00110011 \times 2^{-3}$
 12 times (Lecture #6) 1
 $1 + 4 \times 12 = 49 \leftarrow \text{close to } 52$

this is the answer

no error introduced so 1 is a machine number

*floating means you convert it to binary

*truncate to the 52nd place

Example of machine number: 0.1, 0.5, 0.25

sign $\rightarrow \pm 1.b_1 b_2 \dots b_{52} \times 2^e$ normalized $\Rightarrow$ must be non zero

non machine numbers: $\frac{1}{3}, 0.1, 0.2 \rightarrow$ repeating values $\rightarrow$ infinite binary expansion

$\rightarrow$ smallest number you can represent

51 zeros

$t = 52$

anything you have to round or truncate is not a machine number

rel. error

the relative error will be no less than epsilon machine

$\frac{1}{4} = \frac{1}{2^2} = 1 \times 2^{-2}$

In fact, ^{0.3} of Example 1 shows us why doing a seemingly simple operation in Python (such as adding 0.1 and 0.2) could result in such a strange answer (i.e. 0.30000000000000004). It's because 0.1 and 0.2 have repeating fractional parts in binary, which are subject to rounding error. Thankfully, this rounding error does not exceed $\epsilon_{\text{mach}} = 2^{-52}$ for each floating-point operation, as the example of 0.2 has shown us (i.e. we only rounded up by a fraction equal to $(0.0 \dots 01)_2$ with a total of 51 zeros between the radix point and the digit 1, which is exactly equal to 2^{-52}).

Floating-Point Arithmetic

Laws of floating-point arithmetic: Let x, y be real numbers.

1. If x is a floating-point number, then $\text{fl}(x) = x$
2. $\text{fl}(x \pm y) = [\text{fl}(x) \pm \text{fl}(y)](1 + \delta)$, $|\delta| \leq \epsilon_{\text{mach}}$
3. $\text{fl}(xy) = [\text{fl}(x) \text{fl}(y)](1 + \delta)$, $|\delta| \leq \epsilon_{\text{mach}}$
4. If $y \neq 0$, then $\text{fl}\left(\frac{x}{y}\right) = \left[\frac{\text{fl}(x)}{\text{fl}(y)}\right](1 + \delta)$, $|\delta| \leq \epsilon_{\text{mach}}$

+ - x ÷ each give an error less than epsilon machine

Example 2. Let x, y , and z be floating-point numbers. Compute the value of $\text{fl}(x(y + z))$.

$$\text{fl}(x(y+z)) = \text{fl}(x) \cdot \text{fl}(y+z) (1 + \delta_1) \quad \delta_i \leq \epsilon_{\text{mach}} \text{ for all } i$$

$$= x \text{fl}(y+z) (1 + \delta_1)$$

$$= x [\text{fl}(y) + \text{fl}(z)] (1 + \delta_1) (1 + \delta_2)$$

$$= x (y+z) (1 + \delta_1) (1 + \delta_2)$$

$$= x (y+z) (1 + \delta_1 + \delta_2 + \delta_1 \delta_2)$$

$$x(y+z) (1 + \delta_3)$$

$$\delta_3 \leq 2\epsilon_{\text{mach}}$$

$$\delta_1, \delta_2 \leq \epsilon_{\text{mach}} = 2^{-16}$$

could never store in a machine so negligible so ignore

Theorem: Let $x_1, x_2, \dots, x_n$ be n floating-points numbers. Then

$$\text{fl}(x_1 + \dots + x_n) - (x_1 + \dots + x_n) \approx x_1 \underbrace{(\delta_1 + \delta_2 + \dots + \delta_n)}_{\leq \epsilon_{\text{mach}}} + x_2 \underbrace{(\delta_2 + \dots + \delta_n)}_{\leq (n-1)\epsilon_{\text{mach}}} + \dots + x_n \underbrace{\delta_n}_{\leq \epsilon_{\text{mach}}}$$

where $|\delta_i| < \epsilon_{\text{mach}}$, for $i = 1, 2, \dots, n$.

the error is going to be $\Theta(n)$

Error estimates for certain floating-point operations

In the theorems that follow, we will calculate the absolute error for certain floating-point operations. Recall that absolute forward error can be defined as $|\hat{y} - y|$, where $\hat{y}$ is the approximate value of the

output, and y is the true value of the output. Here, to be more specific, we replace $\hat{y}$ with $\text{fl}(y)$, since the approximation we are making is a floating-point approximation.

Before we proceed, it should be noted that we will use the notation $|\cdot|$ to represent that the corresponding vector or matrix will have absolute values over each of its elements. So:

$$a = \begin{bmatrix} a_1 \\ \vdots \\ a_n \end{bmatrix} \Rightarrow |a| = \begin{bmatrix} |a_1| \\ \vdots \\ |a_n| \end{bmatrix}, \quad A = \begin{bmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & & \vdots \\ a_{n1} & \cdots & a_{nn} \end{bmatrix} \Rightarrow |A| = \begin{bmatrix} |a_{11}| & \cdots & |a_{1n}| \\ \vdots & & \vdots \\ |a_{n1}| & \cdots & |a_{nn}| \end{bmatrix}$$

In addition, when we use inequalities with these matrices, we mean that they apply elementwise:

$$|A| \leq |B| \Rightarrow |a_{ij}| \leq |b_{ij}|, \quad \forall i, j = 1, \dots, n.$$

Finally, the theorems are as follows:

(each corresponding element)

error introduced in dot product & matrix addition & multiplication

Theorem (Rounding error in dot product computation): Let a, b be n -vectors of floating-point numbers. Then

$$|\text{fl}(a \cdot b) - a \cdot b| \leq \phi(n) \epsilon_{\text{mach}} |a| \cdot |b|,$$

where $|a|$ represents the vector with components $|a_i|$, $\cdot$ represents the dot product, and $\phi(n)$ is a small $O(n^2)$ function of n .

Dot product is $O(n^2)$

Theorem: Let $|M|$ be defined to be the matrix with components $|m_{ij}|$. Let A, B be $n \times n$ floating-point matrices and let c be a floating-point number. Then

1. $\text{fl}(cA) = cA + E, \quad |E| \leq \epsilon_{\text{mach}} |cA|$
2. $\text{fl}(A + B) = (A + B) + E, \quad |E| \leq \epsilon_{\text{mach}} |A + B|$

If A and B are compatible for matrix multiplication, then

3. $\text{fl}(AB) = AB + E, \quad |E| \leq n \epsilon_{\text{mach}} |A| |B| + O(\epsilon_{\text{mach}}^2)$

The usefulness of these rather abstract notions may not be immediately evident. The purpose becomes clear when we consider that every time we use a computer to calculate the dot product, or add/multiply matrices, we are introducing floating-point error. These theorems simply give us estimates of what the worst possible error could be for these calculations. We certainly want the error to overall stay as low as possible, to avoid unstable algorithms that produce inaccurate results (see Lecture 4 for more discussion on stability and accuracy).

21-671: Computational Linear Algebra

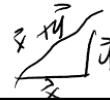
Lecture #7

Updated September 11, 2025

Vector and Matrix Norms

Definition: A **vector norm** $\|\cdot\|: \mathbb{R}^n \rightarrow \mathbb{R}$ is a function that satisfies the following: for all $x, y \in \mathbb{R}^n$:

1. $\|x\| \geq 0$, $\|x\| = 0$ if and only if $x = 0$
2. $\|\alpha x\| = |\alpha| \|x\|$, $\alpha \in \mathbb{R}$
3. $\|x + y\| \leq \|x\| + \|y\|$ (triangle inequality)



Example 1. Here are a few examples of norms for a generic vector $x \in \mathbb{R}^n$, along with specific examples with the vector $a = [1 \ 2 \ 3]^T \in \mathbb{R}^3$.

ℓ^1 norm: $\|x\|_1 = \sum_{i=1}^n |x_i|$ $\rightarrow$ adding up each value of $\vec{a}$ $\|a\|_1 = 6$

ℓ^2 norm: $\|x\|_2 = \left(\sum_{i=1}^n |x_i|^2 \right)^{1/2}$ $\rightarrow$ add up the squares & take root them $\|a\|_2 = \sqrt{14} \approx 3.74$

ℓ^p norm: $\|x\|_p = \left(\sum_{i=1}^n |x_i|^p \right)^{1/p}$ $(1 \leq p < \infty)$ $\|a\|_5 = \sqrt[5]{276} \approx 3.08$

ℓ^∞ norm: $\|x\|_\infty = \max_{1 \leq i \leq n} |x_i|$ $\|a\|_\infty = 3$

A helpful illustration of what norms mean is in the figure to the right. The usual Euclidean (geometrical) distance between two points x and y in space is given by $\|x - y\|_2$. If we set y to be the origin (i.e. $y = 0$) and we look at all points that have a Euclidean distance of 1 (i.e. $\|x\|_2 = 1$) from the origin, then we will create a circle (blue).

However, if we change p from 2 to 1, the “circle” produced by the equation $\|x\|_1 = 1$ becomes a diamond (purple). To see this, imagine that the only way we are allowed to travel is via roads that go horizontally or vertically in a grid pattern. So, going a distance of 1 only results in going out to the edges of the diamond rather than the full circle from before. This is why the ℓ^1 norm is also called the “city block” or “Manhattan” norm.

If we increase p above 2, the “circles” get larger and larger until they become a square at $p = \infty$. This is how we can visualize the ℓ^p norms.

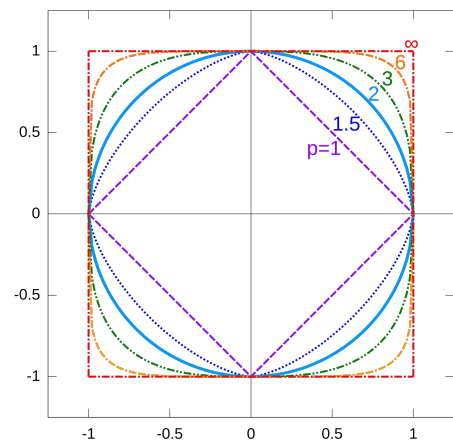


Figure 1: A demonstration of the points which are located a distance of 1 from the origin, in terms of the ℓ^p norms

Hölder's Inequality: Let p, q be such that $1 < p, q < \infty$ and $\frac{1}{p} + \frac{1}{q} = 1$. Then:

$$|x^\top y| \leq \|x\|_p \|y\|_q.$$

Also,

$$|x^\top y| \leq \|x\|_1 \|y\|_\infty.$$

A special case of Hölder's inequality is given when $p = q = 2$, and this is called the **Cauchy-Schwarz inequality**.

$$|x \cdot y| \leq \|x\|_2 \|y\|_2$$

Cauchy-Schwarz

Equivalence of Vector Norms: All vector norms on $\mathbb{R}^n$ are **equivalent**, i.e. for all vector norms $\|\cdot\|_a$ and $\|\cdot\|_b$, there exist positive constants α, β such that

$$\alpha \|x\|_a \leq \|x\|_b \leq \beta \|x\|_a \rightarrow \|x\|_b \leq \|x\|_a \leq \beta_2 \|x\|_b$$

Exercise (just for fun): The equivalence of vector norms is an *equivalence relation*. That is, if we use $\sim$ to denote the equivalence of vector norms, then, for vector norms $\|\cdot\|_a$, $\|\cdot\|_b$, and $\|\cdot\|_c$, we have the following properties:

1. $\|\cdot\|_a \sim \|\cdot\|_a$ ↗ equivalent
2. $\|\cdot\|_a \sim \|\cdot\|_b$ implies $\|\cdot\|_b \sim \|\cdot\|_a$
3. $\|\cdot\|_a \sim \|\cdot\|_b$ and $\|\cdot\|_b \sim \|\cdot\|_c$ implies $\|\cdot\|_a \sim \|\cdot\|_c$

Show that the equivalence of vector norms is an equivalence relation.

Example 2 (Equivalence of vector norms). The following inequalities show the equivalence of certain vector norms:

(a) $\|x\|_2 \leq \|x\|_1 \leq \sqrt{n} \|x\|_2$

(b) $\|x\|_\infty \leq \|x\|_2 \leq \sqrt{n} \|x\|_\infty$

(c) $\|x\|_\infty \leq \|x\|_1 \leq n \|x\|_\infty$

1 & 2 norm are equivalent

∞ & 2 norm are equivalent

1 norm & ∞ norm are equivalent

Matrix Norms

Let $A \in \mathbb{R}^{m \times n}$. The matrix norm $\|\cdot\|_p$ (induced by the vector norm $\|\cdot\|_p$) is given by

$$\|A\|_p = \max_{x \neq 0} \frac{\|Ax\|_p}{\|x\|_p} = \max_{\|x\|_p=1} \|Ax\|_p$$

↗ A by a vector x

Note: We have that $\|Ax\|_p \leq \|A\|_p \|x\|_p$, as $\|A\|_p \geq \frac{\|Ax\|_p}{\|x\|_p}$ for $x \neq 0$. In fact, the induced matrix norm $\|A\|_p$ is the smallest number c such that $\|Ax\|_p \leq c \|x\|_p$.

↗ $\exists$ latin for "and"

↖ $\&$ ampersand

↘ "and for se and"

↘ when reciting the alphabet

A non-induced norm for matrices is known as the **Frobenius norm**, given by

$$\|A\|_F = \left(\sum_{i=1}^m \sum_{j=1}^n |a_{ij}|^2 \right)^{1/2} = \sqrt{A:A},$$

where $A:A$ represents the **Frobenius inner product**, which is defined as

↳ take each element & multiply by each other

$$A:A = \text{tr}(A^T B) = \sum_{i=1}^m \sum_{j=1}^n a_{ij} b_{ij}.$$

The Frobenius inner product may be thought of as the dot product, but for matrices. It is the same as entry-wise multiplication of matrices. For example:

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} : \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} = 1 \cdot 5 + 2 \cdot 6 + 3 \cdot 7 + 4 \cdot 8$$

In general, matrix norms satisfy the same properties as vector norms.

Example 3 (Common matrix norms).

$$\|A\|_1 = \max_{1 \leq j \leq n} \|a_{*j}\|_1 = \max_{1 \leq j \leq n} \sum_{i=1}^m |a_{ij}| \quad (\text{maximum absolute column sum})$$

$$\|A\|_\infty = \max_{1 \leq i \leq m} \|a_{i*}\|_1 = \max_{1 \leq i \leq m} \sum_{j=1}^n |a_{ij}| \quad (\text{maximum absolute row sum})$$

$$\|A\|_2 = \max_{x \neq 0} \frac{\|Ax\|_2}{\|x\|_2} = \sqrt{\lambda_{\max}(A^T A)} \quad (\lambda_{\max}(M) \text{ is the longest eigenvalue of } M)$$

Sometimes $\|A\|_2$ is called the **spectral norm** of $A \in \mathbb{R}^{n \times n}$.

While is easy to compute $\|A\|_1$ and $\|A\|_\infty$ (these can be done in $\mathcal{O}(n^2)$ operations), $\|A\|_2$ is much more complicated. We can find an upper bound on $\|A\|_2$ in $\mathcal{O}(n^2)$ operations, however.

computing $\|A\|_2$ is hard but computing an upper bound can be done in $\mathcal{O}(n^2)$ time

Theorem: Let $A \in \mathbb{R}^{m \times n}$. Then

$$\|A\|_2 \leq \sqrt{\|A\|_1 \|A\|_\infty}$$

Example 4. Let $A = \begin{bmatrix} 2 & -1 & 1 \\ 1 & 0 & 1 \\ 3 & -1 & 4 \end{bmatrix}$. Compute $\|A\|_1$, $\|A\|_\infty$, $\|A\|_F$, and bounds for $\|A\|_2$.

$$\|A\|_1 = \text{maximum} \{2+1+3, |-1|+0+|-1|, 1+1+4\} = \max\{6, 2, 6\} = 6$$

$$\|A\|_\infty = \max\{2+1+1, 1+0+1, 3+1+4\} = \max\{4, 2, 8\} = 8$$

$$\|A\|_F = \sqrt{\begin{matrix} 2^2+1^2+1^2 \\ 1^2+0^2+1^2 \\ 3^2+1^2+4^2 \end{matrix}} = \sqrt{\begin{matrix} 4+1+1 \\ 1+0+1 \\ 9+1+16 \end{matrix}} = \sqrt{34}$$

$$\|A\|_2 \leq \sqrt{\|A\|_1 \|A\|_\infty} = \sqrt{6 \cdot 8} = \sqrt{48} \quad \hookrightarrow \text{upper bound}$$

Just as is the case with vector norms, matrix norms can also be equivalent.

Example 5 (Equivalence of matrix norms). Let $A \in \mathbb{R}^{m \times n}$. Then

- (a) $\|A\|_2 \leq \|A\|_F \leq \sqrt{\min\{m, n\}} \|A\|_2$
- (b) $\max_{i,j} |a_{ij}| \leq \|A\|_2 \leq \sqrt{mn} \max_{i,j} |a_{ij}|$
- (c) $\frac{1}{\sqrt{n}} \|A\|_\infty \leq \|A\|_2 \leq \sqrt{m} \|A\|_\infty$ *this is a norm too*
- (d) $\frac{1}{\sqrt{m}} \|A\|_1 \leq \|A\|_2 \leq \sqrt{n} \|A\|_1$

Definition: A matrix norm $\|\cdot\|_a$ is **submultiplicative** or **consistent** if it satisfies $\|AB\|_a \leq \|A\|_a \|B\|_a$.

Lemma: The induced matrix norm $\|\cdot\|_p$, $1 \leq p < \infty$, is consistent.

Example 6. Using the submultiplicativity of matrices, give an upper bound for $\|A^2\|_2$, where

$$A = \begin{bmatrix} 2 & -1 & 1 \\ 1 & 0 & 1 \\ 3 & -1 & 4 \end{bmatrix} \quad \swarrow \text{example 4}$$

$$\|A^2\|_2 = \|AA\|_2 \leq \|A\|_2 \|A\|_2 \leq \sqrt{48} \sqrt{48}$$

$$\text{submultiplicativity} = 48$$

$\hookrightarrow$ upper bound of $\|A^2\|_2$ is 48

21-671: Computational Linear Algebra

Lecture #8

Updated September 14, 2025

Conditioning of Linear Systems

First, we will examine the condition number of the task of matrix multiplication. Let $A \in \mathbb{R}^{m \times n}$ and let $x \in \mathbb{R}^n$. We want to estimate the condition number of performing the matrix-vector multiplication Ax . Consider $\hat{x} = x + \delta x$, where δx is a small perturbation of x . As the condition number is the ratio of the relative forward error to the relative backward error, for matrix-vector multiplication we have

$$\begin{aligned}\kappa &= \max_{\delta x} \frac{\|A(x + \delta x) - Ax\|}{\|Ax\|} \bigg/ \frac{\|\delta x\|}{\|x\|} \\ &= \max_{\delta x} \frac{\|A(\delta x)\|}{\|\delta x\|} \cdot \frac{\|x\|}{\|Ax\|} = \|A\| \cdot \frac{\|x\|}{\|Ax\|}\end{aligned}$$

Handwritten notes: $\delta x = \hat{x} - x$, relative backward error, definition of $\|A\|$

If A is square and nonsingular, then since

A is invertible

$$\|A^{-1}\| = \max_{y \neq 0} \frac{\|A^{-1}y\|}{\|y\|} \geq \frac{\|A^{-1}(Ax)\|}{\|Ax\|} = \frac{\|x\|}{\|Ax\|},$$

Handwritten notes: max over all $y \neq 0$, plug in $y = Ax$ ($x \neq 0$)

we have

$$\kappa = \|A\| \cdot \frac{\|x\|}{\|Ax\|} \leq \|A\| \|A^{-1}\|.$$

Now, it is certainly true that $Ax = b$ for some $b \in \mathbb{R}^n$. And so κ is the condition number of computing b given x . Conversely, we have that the condition number of **solving** $Ax = b$ is similar to the condition number of computing x given b ($x = A^{-1}b$). Using similar reasoning to the above we see that:

$$\kappa = \|A^{-1}\| \cdot \frac{\|b\|}{\|A^{-1}b\|} \leq \|A^{-1}\| \|A\|.$$

So the condition numbers of computing Ax , and of solving $Ax = b$, are bounded above by $\|A^{-1}\| \|A\|$.

The Condition Number of a Matrix

We will now explore the related concept of condition number of a matrix A itself. It is very convenient to define the condition number of the matrix A as $\|A\| \|A^{-1}\|$, so that it can act as an upper bound for the condition number of computing Ax and the condition number of solving $Ax = b$.

Handwritten notes: upper bound for C.N. of solving $Ax=b$ or computing $Ax=b$

Definition: The **condition number** of the matrix A with regard to the norm $\|\cdot\|_a$, written as $\text{cond}_a(A)$, is given by:

$$\text{cond}_a(A) = \|A\|_a \|A^{-1}\|_a$$

Handwritten note: ★ worst case scenario

$$I = \begin{bmatrix} 1 & \dots & 0 \\ 0 & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & \dots & 0 & 1 \end{bmatrix}$$

Properties of the Condition Number of a Matrix:

1. $\text{cond}_a(I) = 1$, $\text{cond}_a(A) \geq 1$, $\forall A \in \mathbb{R}^{n \times n}$
2. $\text{cond}_a(\alpha A) = \text{cond}_a(A)$, $\forall \alpha \in \mathbb{R}$
3. $\text{cond}_a(AB) \leq \text{cond}_a(A) \text{cond}_a(B)$, $\forall A \in \mathbb{R}^{m \times n}, B \in \mathbb{R}^{n \times m}$
4. $\text{cond}_1(A) = \text{cond}_\infty(A^\top)$

Henceforth, when we speak in general of the condition number, we shall write $\text{cond}(A)$ for the condition number of A with regards to the arbitrary norm $\|\cdot\|$.

Now, we have the following useful theorem regarding a bound on the relative size of perturbations of x given a perturbation of b :

Theorem: If δx and δb are perturbations of $b \neq 0$ and $x \neq 0$, respectively, then if $Ax = b$ and A is nonsingular, we have

$$\frac{\|\hat{x} - x\|}{\|x\|} \quad \text{backward error}$$

$$\frac{\|\delta x\|}{\|x\|} \leq \text{cond}(A) \frac{\|\delta b\|}{\|b\|}$$

$$\frac{\|A\hat{x} - Ax\|}{\|Ax\|} \quad \text{forward error to compute } Ax$$

to compute Ax

In other words, if we want to solve $Ax = b$, any perturbation that happens to the solution x will depend on how much you perturb the right hand side b . We will now give a proof of this result.

Proof of the theorem. Since $Ax = b$ and $A(x + \delta x) = b + \delta b$, then $A\delta x = \delta b$, so $\delta x = A^{-1}(\delta b)$. Then

$$\|\delta x\| \leq \|A^{-1}\| \|\delta b\|$$

prop. of matrix norms

depending how bad your forward error is, that'll determine how much your x changes

Now, $\|Ax\| = \|b\|$, so $\|b\| \leq \|A\| \|x\|$. Hence

$$\frac{1}{\|x\|} \leq \frac{\|A\|}{\|b\|}$$

therefore

$$\frac{\|\delta x\|}{\|x\|} \leq \|A^{-1}\| \|\delta b\| \cdot \frac{\|A\|}{\|b\|},$$

which, rearranged, is the same as

$$\frac{\|\delta x\|}{\|x\|} \leq \|A\| \|A^{-1}\| \frac{\|\delta b\|}{\|b\|}$$

Example 1. Consider the system

$$x_1 + 2x_2 = 3$$

$$2x_1 + 3.999x_2 = 5.999$$

This can be rewritten as the matrix equation $Ax = b$, where

$$A = \begin{bmatrix} 1 & 2 \\ 2 & 3.999 \end{bmatrix}, \quad b = \begin{bmatrix} 3 \\ 5.999 \end{bmatrix}$$

Solving for x

If we perturb the y -coordinate of b by adding .001, then:

- (a) What is the relative error (in terms of the ℓ^2 norm) in b caused by this perturbation? What is the resulting relative error in x ? Is the system $Ax = b$ ill-conditioned or well-conditioned? → how much are you changing your b by?
- (b) Show that the theorem above is respected with regards to these errors.

a) perturb y -coordinate by adding 0.001

$$\delta b = \begin{bmatrix} 0 \\ 0.001 \end{bmatrix}$$

$$\text{rel. error in } b = \frac{\|\delta b\|_2}{\|b\|_2} = \frac{\sqrt{0^2 + 0.001^2}}{\sqrt{3^2 + 5.999^2}} \approx \frac{0.001}{6.7073} \approx 1.4909 \times 10^{-4} \approx 0.015\%$$

$\frac{\|\delta x\|_2}{\|x\|_2}$, need to find δx . $\delta x = \hat{x} - x$. we know neither

Find x :

$$\begin{bmatrix} 1 & 2 \\ 2 & 3.999 \end{bmatrix} x = \begin{bmatrix} 3 \\ 5.999 \end{bmatrix} \rightarrow x = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

Find $\hat{x}$:

$$\begin{bmatrix} 1 & 2 \\ 2 & 3.999 \end{bmatrix} \hat{x} = \hat{b} = b + \delta b = \begin{bmatrix} 3 \\ 5.999 \end{bmatrix} + \begin{bmatrix} 0 \\ 0.001 \end{bmatrix} = \begin{bmatrix} 3 \\ 6 \end{bmatrix}$$

$$\frac{\|\delta x\|_2}{\|x\|_2} = \frac{\|\hat{x} - x\|_2}{\|x\|_2} = \frac{\left\| \begin{bmatrix} 3 \\ 0 \end{bmatrix} - \begin{bmatrix} 1 \\ 1 \end{bmatrix} \right\|_2}{\left\| \begin{bmatrix} 1 \\ 1 \end{bmatrix} \right\|_2} = \frac{\sqrt{2^2 + 1^2}}{\sqrt{1^2 + 1^2}} = \frac{\sqrt{5}}{\sqrt{2}} \approx 1.58 \approx 158\%$$

So, a 0.015% perturbation in b caused a 158% perturbation in the sol. x so solving $Ax = b$ is ill-conditioned

And so it is clear that if $\text{cond}(A)$ is large, even a small perturbation in b can have a drastic effect on the solution. In general, if $\text{cond}(A) \approx 10^k$, then the computed solution to $Ax = b$ will lose around k decimal digits of accuracy! To see why, consider a problem where b is computed to d decimal digits of accuracy. Then the maximum perturbation will be 10^{-d} . If the condition number is 10^k , then by the theorem above, we have that

$$\frac{\|\delta x\|}{\|x\|} \leq \text{cond}(A) \frac{\|\delta b\|}{\|b\|} = 10^k 10^{-d} = 10^{-(d-k)},$$

i.e. the maximum perturbation on x will be $d - k$ digits.

b) Note that the $\text{cond}_2(A) \approx 2.4992 \times 10^4$
 $\frac{\|\delta x\|_2}{\|x\|_2} \approx 1.58$ and
 we want $1.58 \leq \text{cond}_2(A) \frac{\|\delta b\|_2}{\|b\|_2} \approx$
 $2.4992 \times 10^4 \cdot 1.4909 \times 10^{-4}$
 ≈ 3.724 OK

Condition Number and Singularity

The condition number of a matrix is also useful in telling us information about whether a matrix is singular (i.e. A^{-1} does not exist, equivalent to $\det A = 0$). If $\text{cond}(A)$ is large, then A is nearly singular, and $\text{cond}(A)$ is usually a better indicator of near-singularity than $\det(A)$. Similarly, the smallness of $\det(A)$ does not necessarily imply that A is nearly singular. The following two examples illustrate this.

Example 2. If $A = \begin{bmatrix} 10^{10} & 0 \\ 0 & 10^{-10} \end{bmatrix}$, then $\det(A) = 1$. But this matrix is nearly singular, seeing that one of its diagonal entries is nearly zero, while the other is very large. To see this more clearly, note that $\text{cond}_1(A) = \|A\|_1 \|A^{-1}\|_1 = (10^{10})(10^{10}) = 10^{20}$, which is incredibly large. This indicates that the matrix is nearly singular.

→ would lose 20 digits of accuracy solving $Ax=b$

Example 3. Let $A \in \mathbb{R}^{n \times n}$ such that $A = \text{diag}(10^{-k}, \dots, 10^{-k})$. Then $\det(A) = 10^{-nk}$, which is a very small number. But A is actually *not* nearly singular, because its diagonal entries are all equal, meaning that this matrix is just a multiple of the identity matrix, which is far from singular. Indeed, we see that $\text{cond}_1(A) = \|A\|_1 \|A^{-1}\|_1 = 10^{-k} 10^k = 1$, indicating the non-singularity of A .

$$\begin{bmatrix} 10^{-k} & & \\ & 10^{-k} & \\ & & \ddots \\ & & & 10^{-k} \end{bmatrix} = 10^{-k} I$$